

13-数据结构与算法

数据结构与算法面试题

1. 数据结构

问：常见数据结构及特点？

答：

结构	特点	时间复杂度（查找/插入/删除）
数组	连续内存，随机访问	$O(1)$ / $O(n)$ / $O(n)$
链表	离散内存，动态大小	$O(n)$ / $O(1)$ / $O(1)$
栈	LIFO	$O(1)$
队列	FIFO	$O(1)$
哈希表	键值映射	$O(1)$ 平均
二叉树	层次结构	$O(\log n) \sim O(n)$
堆	完全二叉树，极值在根	插入删除 $O(\log n)$
图	节点 + 边	视算法而定

问：数组和链表的区别？

答：

对比	数组	链表
内存	连续	离散
大小	固定（静态）或动态扩容	动态
随机访问	$O(1)$	$O(n)$
插入删除	$O(n)$	$O(1)$ （已知位置）
缓存	友好	不友好

问：为什么数组随机访问是 $O(1)$ ？

答：数组元素在内存中连续存储，已知首地址 `base` 和元素大小 `size`：

地址 = base + index × size

CPU 一次计算即可定位，无需遍历。

链表节点离散分布，只能从头指针逐个 next 查找，故为 $O(n)$ 。

注意：数组插入/删除需移动后续元素，仍为 $O(n)$ 。

问：如何用两个栈实现队列？

答：

```
入队 → push 到 stackIn
出队 → 若 stackOut 空, 将 stackIn 全部 pop 并 push 到 stackOut
      再从 stackOut pop
```

均摊 $O(1)$ ：每个元素最多入栈出栈各 2 次。

变体：两个队列实现栈 — 入栈时把新元素入 q2，再把 q1 元素依次入 q2，交换 q1/q2。

问：二叉搜索树、AVL、红黑树的区别？

答：

树	特点	平衡
BST	左 < 根 < 右	可能退化为链表 $O(n)$
AVL	严格平衡，任意节点左右子树高度差 ≤ 1	查询快，插入删除旋转多
红黑树	近似平衡，5 条性质约束	插入删除快，Java TreeMap、HashMap 用

B/B+ 树：多路平衡，适合磁盘 IO（MySQL 索引）。

问：二叉搜索树（BST）底层操作？为什么会退化？

答：性质：左子树所有 key < 根 < 右子树所有 key。

查找：从根比较，小走左、大走右， $O(h)$ ，h 为树高。

插入：查找到空位置插入新节点。

删除（三种情况）：

```
1. 叶子节点 → 直接删
2. 只有一个子节点 → 用子节点替换
3. 有两个子节点 → 找中序后继（右子树最小）或前驱替换，再删后继
```

退化：有序插入时退化为链表， $h = n$ ，操作变 $O(n)$ 。**解决：**AVL（严格平衡）、红黑树（近似平衡）、B+ 树（多路降低高度）。

问：AVL 树旋转原理？

答：任意节点左右子树高度差 ≤ 1 ，失衡时旋转恢复。

类型	形态	操作
LL	左子树的左子过重	右旋
RR	右子树的右子过重	左旋
LR	左子树的右子过重	先左旋左子，再右旋根
RL	右子树的左子过重	先右旋右子，再左旋根

查询： $O(\log n)$ ，比红黑树略快（更平衡）。插入/删除：旋转次数多，维护成本高 $\rightarrow$ Java TreeMap 选红黑树而非 AVL。

问：B 树和 B+ 树有什么区别？

答：

对比	B 树	B+ 树
数据存储	所有节点存数据	仅叶节点存数据
非叶节点	存 key + 数据	只存索引 key
叶节点链接	无	双向链表，范围查询快
查找	可能在非叶节点终止	必须到叶节点
树高	相对较高	更低，磁盘 IO 更少

MySQL InnoDB：B+ 树，叶节点存完整行数据（聚簇索引）或主键（二级索引）。

InnoDB 页结构（默认 16KB）：

非叶节点：只存索引 key + 子页指针（不存行数据） $\rightarrow$ 单页能存更多 key $\rightarrow$ 树高低
叶节点：存 key + 行数据，叶节点间双向链表 $\rightarrow$ 范围查询顺序扫描

分裂/合并：– 插入满页 $\rightarrow$ 页分裂，中间 key 上推到父节点 – 删除导致页利用率过低 $\rightarrow$ 与兄弟页合并或 redistribution

为什么用 B+ 不用 B：叶节点链表使范围查询无需回溯；非叶不存数据，同样高度能存更多 key，减少磁盘 IO。

问：跳表是什么？和红黑树对比？

答：跳表：有序链表 + 多层索引，上层是下层的「快车道」。

对比	跳表	红黑树
平均复杂度	$O(\log n)$	$O(\log n)$
实现	简单	旋转/变色复杂
空间	多层指针，略高	较低
范围查询	链表遍历方便	中序遍历

应用：Redis ZSet (score 相同时按 member 字典序，底层 ziplist 或 skiplist)。

底层实现：

1. 底层是有序链表 (level 0)，每个节点含 forward[] 多层指针
2. 插入时随机层数：level = 1 + 几何分布 (Redis $p=0.25$ ，平均 $1/(1-p)$ 层)
3. 查找：从最高层开始，向右直到下一跳过大，再下降一层
4. 插入/删除：查找位置后更新各层指针
5. 期望层数 $O(\log n)$ ，期望比较次数 $O(\log n)$

为什么 Redis ZSet 用跳表不用红黑树：实现简单、范围查询方便（链表）、并发下锁粒度更细（可只对部分层加锁）。

问：前缀树 (Trie) 是什么？有什么应用？

答：利用字符串公共前缀减少比较，每个节点代表一个字符。

应用：– 搜索引擎自动补全 – 拼写检查 – IP 路由最长前缀匹配 – 词频统计（节点存计数）

复杂度：插入/查找 $O(m)$ ， m 为字符串长度。

问：堆的实现和应用？

答：完全二叉树，用数组存储。大顶堆：父 $\geq$ 子；小顶堆：父 $\leq$ 子。

操作：– 插入：尾部加入，上浮 $O(\log n)$ – 删除堆顶：尾部换到顶，下沉 $O(\log n)$

应用：优先队列、TopK 问题、堆排序、定时器。

数组存储原理 (0-indexed)：

```

父节点 index = (i - 1) / 2
左子 index   = 2i + 1
右子 index   = 2i + 2

```

完全二叉树性质保证数组无浪费；末尾插入后上浮，删堆顶后末尾换到顶再下沉。

问：并查集 (Union-Find) ？

答：处理 动态连通性 问题（如省份数量、冗余连接）。

优化：– 路径压缩 – 按秩合并

均摊 $O(a(n)) \approx O(1)$ 。

问：哈希表原理？哈希冲突怎么解决？

答：原理： `hash(key) % capacity` 定位桶（slot）。

冲突解决：

方式	说明	代表
链地址法	同桶用链表/红黑树	Java HashMap
开放寻址	冲突后探测下一空位	ThreadLocalMap
再哈希	换 hash 函数重算	较少单独用

负载因子：元素数/桶数，超过阈值（HashMap 0.75）则扩容 rehash。

注意：key 需正确实现 `hashCode()` 和 `equals()`。

问：哈希函数怎么设计？hashCode 和 equals 有什么关系？

答：好的哈希函数：

要求	说明
确定性	同一 key 每次 hash 结果相同
均匀分布	减少冲突，各桶负载均衡
高效	计算快， $O(1)$
雪崩效应	输入微小变化 → 输出大幅变化（密码学 hash 更严格）

hashCode 与 equals 契约（Java）：

```
equals 相等 → hashCode 必须相等
hashCode 相等 → equals 不一定相等（冲突）
hashCode 不等 → equals 一定不等
```

违反契约会导致 HashMap/HashSet 查找不到已存在元素。

String hashCode： `h = 31 * h + char`（31 是奇质数，减少冲突且乘法可优化为移位）。

问：开放寻址法底层原理？和链地址法对比？

答：冲突时在数组内探测下一个空位，无需额外链表节点。

探测方式	公式	特点
线性探测	<code>i, i+1, i+2...</code>	实现简单，易聚集 (primary clustering)
二次探测	<code>i, i+1², i+2²...</code>	减轻聚集
双重哈希	<code>i, i+h2, i+2h2...</code>	第二个 hash 作步长，分布更好

删除：不能直接置空（会截断探测链），需打删除标记（tombstone）。

对比	链地址法	开放寻址
结构	数组 + 链表/树	纯数组
冲突处理	挂链表	探测下一槽
负载因子	可 > 1（链表延伸）	必须 < 1，通常 ≤ 0.7
缓存	链表节点不连续	CPU Cache 友好（连续内存）
代表	HashMap、Redis dict	ThreadLocalMap、Python dict（旧版）

问：HashMap 底层完整原理（JDK 8）？

答：结构：Node[] table 数组 + 链表 + 红黑树。

put 完整流程：

```
1. table 为空 → resize 初始化（默认 16）
2. hash = key.hashCode() ^ (key.hashCode() >>> 16) // 高 16 位参与运算
3. index = (n - 1) & hash // n 为 2 的幂
4. 桶为空 → 直接放 Node
5. 桶不为空：
  a. 头节点 hash 相同且 key.equals → 覆盖 value
  b. 红黑树 → treeify 插入
  c. 链表 → 尾插；若长度 ≥ 8 且 table.length ≥ 64 → 树化
6. size > threshold (capacity × 0.75) → 扩容 2 倍
```

扰动函数 `^ (h >>> 16)`：key.hashCode() 高 16 位与低 16 位异或，使高位也参与 index 计算。因为 `(n-1) & hash` 在 n 较小时只用到了 hash 低位，高位信息浪费，导致冲突增加。

扩容 rehash：

```
新 table = 旧容量 × 2
遍历旧 table 每个桶：
  链表节点：新 index = hash & (newCap - 1)
  JDK 8 优化：节点要么在原 index，要么在 index + oldCap（看 hash 新增 bit 是 0 还是 1）
  无需重新计算 hash，链表可拆成两条
  红黑树：同理拆分或退化
```

树化条件：链表长度 ≥ 8 且数组长度 ≥ 64（否则先扩容不树化）。退化条件：红黑树节点 ≤ 6 时退化为链表。

为什么容量是 2 的幂： $(n-1) \& hash$ 等价于 $hash \% n$ 且更快；二进制全 1 使 hash 各位都能影响 index。

为什么链表 > 8 才树化：Poisson 分布，正常 hash 下链表达 8 的概率约 0.00000006，说明 hash 分布极差或遭受攻击；树化是兜底优化最坏 $O(n) \rightarrow O(\log n)$ 。

问：Redis 字典（dict）底层和 Java HashMap 有何异同？

答：

对比	Java HashMap	Redis dict
结构	数组 + 链表 + 红黑树	数组 + 链表（无树化）
冲突	链地址法	链地址法
扩容	2 倍，一次性 rehash	渐进式 rehash（双 table 逐步迁移）
负载因子	0.75	根据 used/buckets 触发
hash 算法	Object.hashCode 扰动	SipHash（防 Hash 攻击）

Redis 渐进式 rehash：维护 `ht[0]` 和 `ht[1]`，新写入进 `ht[1]`，每次操作迁移若干 bucket，迁移完切换。避免一次性 rehash 阻塞单线程。

问：布隆过滤器原理？有什么局限？

答：位数组 + 多个 Hash 函数：

添加：各 hash 位置置 1
查询：各 hash 位全为 1 → 可能存在；任一为 0 → 一定不存在

特点	说明
优点	空间小、查询快
假阳性	可能存在但实际不存在（误判）
无假阴性	说不存在则一定不存在
不支持删除	需 Counting Bloom Filter

应用：短链防穿透、爬虫 URL 去重、HBase 读过滤。

复杂度：插入/查询 $O(k)$ ，k 为 hash 函数个数。

2. 排序算法

问：常见排序算法及复杂度？

答：

算法	平均	最坏	空间	稳定
冒泡	$O(n^2)$	$O(n^2)$	$O(1)$	是
选择	$O(n^2)$	$O(n^2)$	$O(1)$	否
插入	$O(n^2)$	$O(n^2)$	$O(1)$	是
快排	$O(n \log n)$	$O(n^2)$	$O(\log n)$	否
归并	$O(n \log n)$	$O(n \log n)$	$O(n)$	是
堆排	$O(n \log n)$	$O(n \log n)$	$O(1)$	否
计数	$O(n+k)$	$O(n+k)$	$O(k)$	是

问：快排原理？

答：1. 选 pivot（通常随机或三数取中） 2. 分区：小于 pivot 放左边，大于放右边 3. 递归左右子数组

优化：小数组用插入排序、三数取中选 pivot。

问：什么是排序稳定性？哪些算法稳定？

答：稳定：相等元素排序后相对顺序不变。

稳定	不稳定
冒泡、插入、归并、计数	快排、堆排、选择

意义：多字段排序时，先按次要字段排，再按主要字段稳定排序可保留次序。

问：堆排序原理？稳定吗？

答：

1. 建大顶堆 $O(n)$
 2. 堆顶（最大值）与末尾交换
 3. 堆大小 -1 ，对堆顶下沉 $O(\log n)$
 4. 重复 2~3 直到排序完成

时间： $O(n \log n)$ ，空间 $O(1)$ ，不稳定。

与快排对比：堆排最坏也是 $O(n \log n)$ ，但常数因子大，实际常不如快排快。

问：数据量太大无法一次装入内存怎么排序？

答：外部排序（多路归并）：

1. 分块读入内存，块内快排/堆排
2. 每块写入磁盘临时文件（已有序）
3. 多路归并：小顶堆维护各文件当前最小值
4. 依次输出，得到全局有序

应用：数据库大表排序、MapReduce、日志合并。

应用：数据库大表排序、MapReduce、日志合并。

优化：增大内存块减少 IO 次数；败者树优化多路归并。

问：归并排序和快速排序怎么选？

答：

对比	归并	快排
最坏时间	$O(n \log n)$	$O(n^2)$
空间	$O(n)$	$O(\log n)$
稳定	是	否
实际性能	常数大	通常更快

选型： – 需要稳定排序 → 归并 – 一般场景 → 快排（Java Arrays.sort 基本类型用快排/堆排，对象用 TimSort=归并+插入） – 链表排序 → 归并（空间 $O(1)$ 可原地） – 外部排序 → 多路归并

3. 查找与遍历

问：DFS 和 BFS 的区别？

答：

对比	DFS	BFS
数据结构	栈（递归）	队列
空间	$O(h)$ 树高	$O(w)$ 最宽层
应用	路径、连通性、回溯	最短路径（无权图）、层序遍历

问：二分查找的边界？

答：

```
// 查找第一个 >= target 的位置
int lo = 0, hi = n;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (nums[mid] < target) lo = mid + 1;
    else hi = mid;
}
```

前提：有序数组。注意 `lo < hi` vs `lo <= hi`、mid 计算防溢出。

4. 算法思想

问：动态规划三要素？

答：1. 最优子结构：大问题最优解包含子问题最优解 2. 重叠子问题：子问题重复出现，用表格缓存 3. 状态转移方程：`dp[i] = f(dp[i-1], ...)`

步骤：定义状态 → 转移方程 → 初始条件 → 计算顺序 → 返回结果。

经典题：爬楼梯、背包问题、最长递增子序列、编辑距离。

问：贪心算法适用条件？

答：1. 最优子结构 2. 贪心选择性质：局部最优能导致全局最优

经典题：区间调度、跳跃游戏、分糖果。

注意：贪心不一定正确，需证明或反证。

问：回溯算法模板？

答：

```
void backtrack(路径, 选择列表) {
    if (满足结束条件) { 结果.add(路径); return; }
    for (选择 : 选择列表) {
        做选择;
        backtrack(路径, 选择列表);
        撤销选择;
    }
}
```

经典题：全排列、组合、子集、N 皇后、单词搜索。

5. 高频技巧

问：双指针有哪些类型？

答：1. **对撞指针**：两端向中间（两数之和、盛水） 2. **快慢指针**：链表环检测、找中点 3. **滑动窗口**：子串/子数组问题（最长无重复子串）

问：前缀和的应用？

答：预处理 `prefix[i] = sum(nums[0..i-1])`，区间和 `sum(l, r) = prefix[r+1] - prefix[l]`， $O(1)$ 查询。

扩展：二维前缀和、前缀异或（区间异或）。

问：单调栈的应用？

答：维护单调递增/递减栈，用于下一个更大/更小元素问题。

经典题：– 每日温度（下一个更高温度） – 柱状图最大矩形 – 接雨水

模板：

```
for (int i = 0; i < n; i++) {
    while (!st.isEmpty() && nums[st.peek()] < nums[i]) {
        int idx = st.pop();
        // 处理 idx 的「下一个更大元素」为 i
    }
    st.push(i);
}
```

问：拓扑排序是什么？什么场景用？

答：有向无环图（DAG）的线性排序，使每条边 $u \rightarrow v$ 中 u 在 v 前面。

算法：1. **Kahn (BFS)**：入度为 0 的节点入队，出队时减邻居入度 2. **DFS**：后序栈逆序

应用：课程表、任务调度、编译依赖、Maven 依赖解析。

应用：课程表、任务调度、编译依赖、Maven 依赖解析。

判环：Kahn 无法处理全部节点则存在环。

问：无权图最短路径和有权重图怎么求？

答：

场景	算法	复杂度
无权图最短路径	BFS	$O(V+E)$
非负权图	Dijkstra (小顶堆)	$O(E \log V)$
有负权边	Bellman-Ford	$O(VE)$
全源最短路	Floyd	$O(V^3)$

Dijkstra 核心：贪心，每次取距离最小的未访问节点，松弛其邻居。

注意：Dijkstra 不能处理负权边（需 Bellman-Ford）。

6. 刷题建议

问：算法怎么准备？

答：

必刷清单： – LeetCode Hot 100 – 剑指 Offer – 按标签：数组、链表、二叉树、DP、回溯、图

复习策略： 1. 先学模板（二分、滑动窗口、回溯、DP） 2. 同类型题连续刷 3~5 道 3. 限时 20~30 分钟/题 4. 总结题型和套路

面试常考： – 反转链表 – 二叉树层序/前中后序遍历 – 最长无重复子串 – 三数之和 – LRU 缓存 – 合并 K 个有序链表

7. 红黑树深入

问：红黑树的两条性质？

答： 1. 节点是红色或黑色 2. 根节点是黑色 3. 叶子节点 (NIL) 是黑色 4. 红色节点的子节点必须是黑色（不能有连续红节点） 5. 从任意节点到其每个叶子的所有路径，包含相同数量的黑色节点（黑高相同）

保证： 最长路径不超过最短路径的 2 倍，查找/插入/删除均为 $O(\log n)$ 。

应用： Java TreeMap / HashMap 树化节点、Linux CFS 调度器、epoll 内核红黑树。

问：红黑树左旋和右旋是什么？

答： 右旋（以 y 为轴，解决左子过重）：



x 成为新子树根，y 成为 x 的右孩子，B 转挂到 y 的左子。

左旋：右旋的镜像，解决右子过重。

旋转不改变中序遍历结果（仍有序），只调整结构。

问：红黑树插入后如何修复？

答：新节点默认**红色**（少破坏黑高），若父节点也是红色则违规（连续红）。

三种情况（当前节点 N，父 P，叔 U，祖父 G）：

情况	条件	修复
1	叔节点 U 为红	P、U 变黑，G 变红；G 作为新 N 继续向上修复
2	U 为黑，N 是 P 的 内侧 子	对 P 旋转，转为情况 3
3	U 为黑，N 是 P 的 外侧 子	P 变黑，G 变红，对 G 旋转

修复后根节点强制变黑。

问：红黑树删除如何修复？

答：删除比插入复杂，核心问题是可能**减少某路径的黑节点数**（产生「双黑」节点）。

步骤概要：

- 按 BST 规则删除（0/1/2 子节点情况）
 - 若删的是黑节点，后继替换节点可能破坏黑高
 - 通过旋转 + 变色，向根方向调整「双黑」节点
 - 兄弟节点为红 → 旋转换黑兄弟
 - 兄弟子全黑 → 兄弟变红，双黑上移
 - 兄弟有红子 → 旋转 + 变色，黑高恢复
 - 根节点最终变黑

面试：能说清插入三种情况即可；删除了解「双黑」概念和旋转方向足够。

问：为什么 HashMap 树化用红黑树不用 AVL？

答：

对比	AVL	红黑树
平衡程度	严格（高度差 ≤ 1）	近似（最长 ≤ 2×最短）
查询	略快	略慢
插入/删除	旋转多	旋转少
HashMap 场景	冲突桶偶尔树化， 插入更频繁	 综合更优

HashMap 桶内链表转树后仍有新元素插入，红黑树维护成本低于 AVL；桶内元素通常不多，O(log n) 已足够。

8. TopK 问题

问：TopK 问题的解法有哪些？

答：

方法	时间复杂度	空间	适用
排序	$O(n \log n)$	$O(1)$	数据量小
小顶堆	$O(n \log k)$	$O(k)$	海量数据 TopK
快排 partition	$O(n)$ 平均	$O(1)$	找第 K 大
桶排序	$O(n)$	$O(n)$	数据范围小且密集
分治	$O(n)$	$O(1)$	分布式 TopK

小顶堆求 TopK 大：

维护大小为 K 的小顶堆
遍历：若元素 > 堆顶，弹出堆顶并入堆
最终堆中即为 TopK

第 K 大：快排 partition，左侧比 pivot 大，右侧比 pivot 小，递归一侧。

9. LRU 缓存

问：如何用数据结构设计 LRU 缓存？

答：哈希表 + 双向链表， $O(1)$ get 和 put。

```
class LRUCache {
    // HashMap<key, Node> 快速定位
    // 双向链表：头=最近使用，尾=最久未使用
    int get(int key) {
        if (!map.containsKey(key)) return -1;
        moveToHead(map.get(key));
        return node.val;
    }
    void put(int key, int val) {
        if (map.containsKey(key)) { update; moveToHead; }
        else {
            addToHead;
            if (size > capacity) removeTail;
        }
    }
}
```

Redis LRU：近似 LRU，随机采样 5 个 key 淘汰最久未访问的。

LFU：最不经常使用，需维护访问频率，Redis 4.0+ 支持 LFU 策略。

10. 二叉树遍历

问：二叉树前序、中序、后序、层序遍历？

答：

遍历	顺序	递归	迭代
前序	根左右	简单	栈
中序	左根右	简单	栈（BST 得有序序列）
后序	左右根	简单	栈（或双栈）
层序	逐层	BFS 队列	队列

前序迭代：

```
Stack<Node> st; st.push(root);
while (!st.isEmpty()) {
    Node n = st.pop(); visit(n);
    if (n.right != null) st.push(n.right);
    if (n.left != null) st.push(n.left);
}
```

经典题：根据前序+中序构建树、最大深度、最近公共祖先、路径和。

11. 动态规划

问：动态规划经典题模板？

答：

1. 线性 DP（爬楼梯/打家劫舍）：

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i])
```

2. 背包问题：

```
// 0-1 背包: dp[j] = max(dp[j], dp[j-w[i]] + v[i]), j 逆序
// 完全背包: j 正序
```

3. 最长递增子序列 LIS：

```
// O(n^2): dp[i] = max(dp[j]+1), j<i, nums[j]<nums[i]
// O(n log n): 维护 tails 数组 + 二分
```

4. 编辑距离:

```
dp[i][j] = min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+(s[i]!=t[j]))
```

5. 股票买卖:

```
dp[i][0] = max(dp[i-1][0], dp[i-1][1] + prices[i]) // 不持有
dp[i][1] = max(dp[i-1][1], dp[i-1][0] - prices[i]) // 持有
```

解题步骤: 定义状态 → 转移方程 → 初始化 → 遍历顺序 → 返回值。

12. 时间复杂度分析

问: 如何分析算法时间复杂度?

答:

大 O 表示法: 关注增长趋势, 忽略常数和低阶项。

复杂度	名称	示例
$O(1)$	常数	哈希查找
$O(\log n)$	对数	二分、平衡树
$O(n)$	线性	遍历
$O(n \log n)$	线性对数	快排、归并
$O(n^2)$	平方	双重循环
$O(2^n)$	指数	子集枚举
$O(n!)$	阶乘	全排列

分析方法: 1. 循环: 单层 $O(n)$, 嵌套 $O(n^2)$ 2. 递归: 主定理 $T(n) = aT(n/b) + f(n)$, 快排平均 $O(n \log n)$ 3. 均摊: 动态数组扩容, 单次 $O(n)$ 均摊 $O(1)$ 4. 最坏 vs 平均: 快排最坏 $O(n^2)$, 平均 $O(n \log n)$

空间复杂度: 辅助空间, 递归深度 $O(h)$ 。

13. 更多高频题

问: 如何判断链表有环?

答: 快慢指针: 慢指针走 1 步, 快指针走 2 步, 相遇则有环。

找环入口：相遇后，一个指针回 head，同步走，再次相遇点即入口（数学证明：头到入口 = 相遇点到入口）。

问：如何反转链表？

答：迭代（三指针）：

```
ListNode prev = null, curr = head;
while (curr != null) {
    ListNode next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
}
return prev;
```

递归：先反转子链表，再让 head.next.next = head, head.next = null。

变种：反转前 N 个、K 个一组反转、两两交换。

问：合并 K 个有序链表？

答：小顶堆：K 个链表头入堆，每次弹出最小，将其 next 入堆。O(N log K)，N 为总节点数。

分治：两两合并，O(N log K)。

问：最长无重复子串？

答：滑动窗口 + HashSet/Map 记录字符位置：

```
for (int r = 0; r < n; r++) {
    while (set.contains(s[r])) { set.remove(s[l]); l++; }
    set.add(s[r]);
    ans = max(ans, r - l + 1);
}
```

图解加深

专题	要点
Hash 底层	扰动函数、rehash、树化阈值、开放寻址 vs 链地址
BST/AVL	插入删除、LL/RR/LR/RL 旋转
红黑树	五条性质、左旋右旋、插入三种修复、双黑删除
B+/B 树	页分裂、叶节点链表、InnoDB 聚簇索引
跳表	多层指针、随机层高、Redis ZSet
Trie/布隆	前缀匹配、假阳性
堆	数组下标公式、上浮下沉、TopK
LRU	HashMap + 双向链表
二叉树	四种遍历、递归/迭代
DP	状态定义、背包、LIS、编辑距离
复杂度	大 O、主定理、均摊分析
外部排序	分块 + 多路归并

大厂追问

1. **HashMap 为什么用红黑树不用 AVL?** 红黑树插入删除旋转更少，综合性能更好；AVL 查询略优但维护成本高。
2. **TopK 为什么用小顶堆不是大顶堆?** 维护 K 个最大元素，堆顶是 K 个中最小的，新元素比堆顶大才替换。
3. **LRU 为什么用双向链表?** 删除中间节点 O(1) 需前驱指针，单向链表做不到。
4. **DP 和贪心怎么区分?** DP 需最优子结构+重叠子问题，穷举所有可能；贪心每步局部最优，需证明正确性。
5. **快排最坏什么情况?** 已有序数组 + 选第一个为 pivot，O(n²)；随机 pivot 或三数取中可避免。
6. **100 亿数据找 Top10 万?** 无法一次加载，分片各自 TopK 再合并，或多路归并。
7. **HashMap 链表转红黑树阈值为什么是 8?** Poisson 分布，正常 hash 下链表长度 ≥8 概率极低 (0.00000006)，超阈值说明 hash 攻击或分布极差。
8. **布隆过滤器能删除元素吗?** 标准 Bloom 不能（置 1 可能影响其他元素）；Counting Bloom Filter 用计数器支持删除。
9. **HashMap 扩容时链表为什么可以拆成两条?** JDK 8 看 hash 在新增 bit 上是 0 还是 1，节点要么留原 index，要么到 index+oldCap，无需重新 hash 整个链表。
10. **红黑树插入为什么新节点默认红色?** 黑色会增加黑高，影响所有路径；红色只可能违反「连续红」，局部修复即可。